🧠 TEORÍA: ¿Cómo funciona el Sensor de Ultrasonidos (HC-SR04)?

Antes de conectar cables, es fundamental entender qué estamos usando. El sensor **HC-SR04** tiene 4 pines:

1. **VCC:** Energía (5V).
2. **TRIG (Trigger/Disparador):** El "boca" del sensor.
3. **ECHO (Eco):** El "oído" del sensor.
4. **GND:** Tierra.

El proceso físico:

1. El Arduino envía una señal al pin **Trig**.
2. El sensor lanza un sonido ultrasónico (40kHz, inaudible para humanos) hacia adelante.
3. El sonido viaja, rebota en un objeto (tu mano) y regresa.
4. El pin **Echo** recibe el rebote.
5. **La Matemática:** El Arduino mide el tiempo que tardó el sonido en ir y volver. Sabiendo que la velocidad del sonido es 343 m/s, aplica la fórmula:
   Distancia=2Tiempo×0.034
   (Se divide por 2 porque el sonido fue y volvió).

PARTE 1: SIMULACIÓN EN TINKERCAD

📋 PASO 1: Diseño del Circuito

1. Entra a **Tinkercad Circuits** y crea un nuevo circuito.
2. Busca y arrastra los siguientes componentes:
   - **Arduino Uno R3**.
   - **Placa de pruebas pequeña (Protoboard)**.
   - **Sensor de distancia ultrasónico** (Busca el que tiene **4 pines**, ojo que hay uno de 3).
   - **Piezo** (Zumbador).

📋 PASO 2: Conexiones en el Simulador

Realiza el cableado virtualmente:

A. El Sensor HC-SR04:

- **VCC** → 5V del Arduino.
- **GND** → GND del Arduino.
- **TRIG** → Pin Digital **9**.
- **ECHO** → Pin Digital **10**.

B. El Zumbador (Piezo):

- **Positivo (+)** → Pin Digital **8**.
- **Negativo (-)** → GND.

📋 PASO 3: Programación en Bloques (Lógica Básica)

Nota: Hacer una canción completa como Star Wars solo con bloques es extremadamente largo y tedioso. Aquí haremos la lógica de detección en bloques y luego pasaremos a texto para la música.

1. Ve a la pestaña **Código** y selecciona **Bloques**.
2. Crea una **Variable** llamada `distancia`.

Construcción de los Bloques:

```text

[Bucle Principal]
  1. ┌─ Definir variable [distancia] en:
     └─ [Leer sensor ultrasónico en pasador de desencadenador (9) pasador de eco
(10)]

  2. [Imprimir en monitor serie] variable [distancia] (opcional, para ver los
cm)

  3. ┌─ SI ( [distancia] < 10 ) ENTONCES:  <-- Usamos 10cm para facilitar la
simulación
     │     ┌─ Reproducir altavoz en pasador (8) con nota (50) durante (0.2) seg
     │     └─ (Aquí iría la canción completa)
     └─ SI NO:
           └─ Desactivar altavoz en pasador (8)
```

⚠️ IMPORTANTE PARA LA MÚSICA:

Para que suene la "Marcha Imperial" real, los bloques son muy limitados. En Tinkercad, cambia la vista de "Bloques" a **"Bloques + Texto"** o solo **"Texto"** para pegar el código real de la canción.

PARTE 2: PROYECTO FÍSICO CON ARDUINO MEGA

Ahora vamos al mundo real. El código y las conexiones son compatibles entre el UNO (Tinkercad) y el MEGA (Físico), solo asegúrate de usar los pines correctos.

🛠️ Materiales Necesarios

1. Arduino Mega 2560 (y su cable USB).
2. Sensor HC-SR04.
3. Zumbador Pasivo (Passive Buzzer) - *Importante: Los activos solo pitan, los pasivos pueden hacer notas musicales*.
4. Cables Dupont (Macho-Macho y Macho-Hembra).
5. Protoboard.

📋 PASO 1: Conexiones Físicas en el Arduino Mega

El Arduino Mega es más grande, pero los pines digitales funcionan igual para este ejercicio.

```text

SENSOR ULTRASONIDOS (HC-SR04)      ARDUINO MEGA
   VCC ──────────────────────────────► 5V
   TRIG ─────────────────────────────► Pin 9
   ECHO ─────────────────────────────► Pin 10
```

```
   GND ──────────────────────────────────▶ GND

ZUMBADOR (BUZZER PASIVO)           ARDUINO MEGA
   Pata Larga (+) ──────────────────────▶ Pin 8
   Pata Corta (-) ──────────────────────▶ GND
```

📋 PASO 2: El Código Maestro (Arduino IDE)

Copia y pega este código en tu Arduino IDE. He incluido las frecuencias de las notas y la lógica para detectar la mano a 4cm o menos.

```
/*
 * PROYECTO: ALARMA DE PROXIMIDAD STAR WARS
 * PLACA: Arduino Mega 2560
 * SENSOR: HC-SR04
 */

// 1. DEFINICIÓN DE PINES
const int pinTrig = 9;
const int pinEcho = 10;
const int pinBuzzer = 8;

// Variables para cálculos
long duracion;
int distancia;

// 2. DEFINICIÓN DE NOTAS MUSICALES (Frecuencias en Hz)
#define NOTE_A3  220
#define NOTE_AS3 233
#define NOTE_B3  247
#define NOTE_C4  261
#define NOTE_CS4 277
#define NOTE_D4  294
#define NOTE_DS4 311
#define NOTE_E4  330
#define NOTE_F4  349
#define NOTE_FS4 370
#define NOTE_G4  392
#define NOTE_GS4 415
#define NOTE_A4  440
#define NOTE_AS4 466
#define NOTE_B4  494
#define NOTE_C5  523
#define NOTE_CS5 554
#define NOTE_D5  587
#define NOTE_DS5 622
#define NOTE_E5  659
#define NOTE_F5  698
#define NOTE_FS5 740
#define NOTE_G5  784
#define NOTE_GS5 831
#define NOTE_A5  880

// 3. CONFIGURACIÓN INICIAL
void setup() {
  pinMode(pinTrig, OUTPUT); // El pin Trig emite sonido
  pinMode(pinEcho, INPUT);  // El pin Echo escucha
  pinMode(pinBuzzer, OUTPUT);

  Serial.begin(9600); // Para ver la distancia en el monitor serie
```

```
  Serial.println("=== SISTEMA INICIADO: ACERQUE SU MANO ===");
}

// 4. BUCLE PRINCIPAL
void loop() {
  // --- A. LIMPIAR EL SENSOR ---
  digitalWrite(pinTrig, LOW);
  delayMicroseconds(2);

  // --- B. EMITIR PULSO (DISPARO) ---
  digitalWrite(pinTrig, HIGH);
  delayMicroseconds(10);
  digitalWrite(pinTrig, LOW);

  // --- C. LEER EL TIEMPO DE REBOTE ---
  duracion = pulseIn(pinEcho, HIGH);

  // --- D. CALCULAR DISTANCIA (cm) ---
  distancia = duracion * 0.034 / 2;

  // Mostrar en monitor serie para calibrar
  Serial.print("Distancia: ");
  Serial.print(distancia);
  Serial.println(" cm");

  // --- E. CONDICIÓN DE ACTIVACIÓN ---
  // Si la distancia es menor o igual a 15 cm (para dar margen,
  // aunque el ejercicio pide 4cm, 15cm es más fácil de probar sin tocar el
sensor)
  // Si quieres estrictamente 4cm, cambia el 15 por un 4.

  if (distancia > 0 && distancia <= 10) {
    Serial.println("¡INTRUSO DETECTADO! - Ejecutando Marcha Imperial");
    tocarMarchaImperial();
    delay(2000); // Esperar 2 segundos antes de volver a medir
  }

  delay(100); // Pequeña pausa para estabilidad
}

// 5. FUNCIÓN DE LA MARCHA IMPERIAL
void tocarMarchaImperial() {
    // Primera sección
    tone(pinBuzzer, NOTE_A4); delay(500); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_A4); delay(500); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_A4); delay(500); noTone(pinBuzzer); delay(50);

    tone(pinBuzzer, NOTE_F4); delay(350); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_C5); delay(150); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_A4); delay(500); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_F4); delay(350); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_C5); delay(150); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_A4); delay(650); noTone(pinBuzzer); delay(50);

    delay(500); // Pausa breve

    // Segunda sección (Agudos)
    tone(pinBuzzer, NOTE_E5); delay(500); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_E5); delay(500); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_E5); delay(500); noTone(pinBuzzer); delay(50);

    tone(pinBuzzer, NOTE_F5); delay(350); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_C5); delay(150); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_GS4); delay(500); noTone(pinBuzzer); delay(50);
```

```
    tone(pinBuzzer, NOTE_F4); delay(350); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_C5); delay(150); noTone(pinBuzzer); delay(50);
    tone(pinBuzzer, NOTE_A4); delay(650); noTone(pinBuzzer); delay(50);
}
```

📋 PASO 3: Cargar y Probar

1. Conecta tu Arduino Mega al PC.
2. En Arduino IDE: **Herramientas** > **Placa** > **Arduino Mega or Mega 2560**.
3. Selecciona el **Puerto** correcto.
4. Dale al botón **Subir** (flecha derecha).
5. Abre el **Monitor Serie** (lupa arriba a la derecha) para ver las medidas de distancia en tiempo real.
6. Acerca tu mano al sensor. Cuando el monitor marque menos de 10cm (o 4cm si lo cambiaste)... **¡Que la fuerza te acompañe!**

⚠️ Solución de problemas comunes

- **Lecturas de 0 cm o erráticas:** Revisa los cables Trig y Echo. A veces el sensor se "bloquea", desconecta y conecta el VCC para reiniciarlo.
- **El sonido es un "click" constante:** Estás usando un buzzer activo o no lo conectaste a un pin PWM (aunque la función `tone()` funciona en casi todos los pines, asegúrate de que sea el pin 8 como en el código).
- **Volumen muy bajo:** Quita la pegatina protectora que traen algunos buzzers encima.

Versión Claude

🎵 Tutorial Completo: Sensor Ultrasónico + Melodía Star Wars

📚 PARTE 1: TEORÍA - ¿Cómo Funciona el Sensor Ultrasónico?

🔬 El Sensor HC-SR04: Explicación Completa

text

```
┌──────────────────────────────────────────────────────────────┐
│                    SENSOR ULTRASÓNICO HC-SR04                │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│          ┌──────────────────────────────────┐                │
│          │                                  │                │
│          │   ┌─────────┐      ┌─────────┐   │                │
│          │   │  )))    │      │   (((   │   │                │
│          │   │ EMISOR  │      │RECEPTOR │   │                │
│          │   │   TX    │      │   RX    │   │                │
│          │   └─────────┘      └─────────┘   │                │
│          │                                  │                │
│          │  VCC  TRIG  ECHO  GND            │                │
│          └──────────────────────────────────┘                │
│              │     │     │     │                             │
│             5V   Pin   Pin   GND                             │
│                 Digital Digital                              │
└──────────────────────────────────────────────────────────────┘
```

🔊 Principio de Funcionamiento: SONAR

text

```
El sensor funciona como un SONAR (igual que los murciélagos o submarinos):

    PASO 1: EMISIÓN                    PASO 2: REBOTE
    ═══════════════                    ══════════════

    Sensor                Objeto       Sensor                Objeto
    ┌─────┐                ┌─┐         ┌─────┐                ┌─┐
    │ E   │──► ))) ))) ))) │ │         │ R   │◄── ((( ((( ((( │ │
    │ R   │                │ │ │       │ X   │                │ │ │
    └─────┘                └─┘         └─────┘                └─┘

    El EMISOR envía              El sonido REBOTA en el
    ondas ultrasónicas           objeto y vuelve al RECEPTOR
    (40,000 Hz - inaudible)
```

```
PASO 3: CÁLCULO DE DISTANCIA
═══════════════════════════════

┌──────────────────────────────────────────────────────────┐
│                                                          │
│  Distancia = (Tiempo × Velocidad del Sonido) ÷ 2         │
│                                                          │
│  Donde:                                                  │
│  • Tiempo = microsegundos que tarda el eco en volver     │
│  • Velocidad del sonido = 343 m/s = 0.0343 cm/µs         │
│  • Se divide entre 2 porque el sonido va Y vuelve        │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

⚡ Los 4 Pines del HC-SR04

```
text
```

PIN	FUNCIÓN
VCC	🔌 ALIMENTACIÓN POSITIVA • Conectar a 5V del Arduino • Proporciona energía al sensor • Consumo: ~15mA
TRIG	📤 TRIGGER (DISPARADOR) • Pin de ENTRADA para el sensor • Arduino envía un pulso de 10µs • Este pulso "dispara" la emisión ultrasónica • Conectar a cualquier pin digital del Arduino
ECHO	📥 ECO (RESPUESTA) • Pin de SALIDA del sensor • Envía un pulso al Arduino • La DURACIÓN del pulso indica la distancia • Pulso largo = objeto lejos • Pulso corto = objeto cerca
GND	⏚ TIERRA (GROUND) • Conectar al GND del Arduino • Cierra el circuito eléctrico

---

📊 Diagrama de Tiempo del Sensor

```
text

SECUENCIA DE MEDICIÓN:
```

```
Señal TRIG (enviada por Arduino):

      ┌──┐
──────┘  └──────────────────────────────────────────
      │10µs│

Arduino envía pulso HIGH de 10 microsegundos


Señal ECHO (recibida del sensor):

                  ┌──────────────────────────┐
──────────────────┘                          └──────────────────
                  │◄── Tiempo (µs) ──►│

El ancho del pulso es PROPORCIONAL a la distancia


EJEMPLO DE CÁLCULO:
═══════════════════

Si el pulso ECHO dura 580 microsegundos:

Distancia = (580 µs × 0.0343 cm/µs) ÷ 2
Distancia = 19.894 ÷ 2
Distancia = 9.95 cm ≈ 10 cm
```

📏 Especificaciones Técnicas del HC-SR04

text

ESPECIFICACIONES HC-SR04	
Voltaje de operación	5V DC
Corriente de operación	15mA
Frecuencia ultrasónica	40 kHz
Rango de medición	2 cm - 400 cm
Precisión	±3 mm
Ángulo de detección	15° - 30°
Dimensiones	45 x 20 x 15 mm

```
CONO DE DETECCIÓN (vista superior):

                    30°
                   /  \
                  /    \
 ┌──────┐        /      \
 │SENSOR│◄──────────────►  Hasta 4 metros
 └──────┘        \      /
                  \    /
                   \  /
```

⚠️ Objetos muy pequeños o en ángulos extremos pueden no ser detectados correctamente

📐 PARTE 2: PROYECTO EN TINKERCAD

📋 PASO 1: Acceder y Crear Proyecto

text

```
1. Abre: https://www.tinkercad.com
2. Inicia sesión
3. Ve a "Circuits" → "Create new Circuit"
4. Nombra tu proyecto: "Star Wars Ultrasonic"
```

📋 PASO 2: Componentes Necesarios

Componente	Cantidad	Búsqueda en Tinkercad
Arduino UNO R3	1	"Arduino"
Breadboard	1	"Breadboard"
Ultrasonic Sensor (HC-SR04)	1	"Ultrasonic"
Piezo Buzzer	1	"Piezo"
LED Rojo (indicador)	1	"LED"
Resistencia 220Ω	1	"Resistor"

📋 PASO 3: Diagrama de Conexiones

text

```
                    ESQUEMA DE CONEXIONES

┌──────────────────────────────────────────────────────────┐
│                        PROTOBOARD                        │
│ − − − − − − − − − − − − − − − − − − − − − − − − − − − − − (GND) │
│ + + + + + + + + + + + + + + + + + + + + + + + + + + + + + (5V) │
│                                                          │
│      ┌──────────────────────────┐                        │
│      │  ┌──────┐  ┌──────┐      │   SENSOR HC-SR04       │
│      │  │  )))  │  │  (((  │      │                        │
│      │  └──────┘  └──────┘      │                        │
│      │  VCC TRIG ECHO GND       │                        │
│      └──────────────────────────┘                        │
│        │    │    │    │                                  │
│        │    │    │    └──────────────► GND               │
│        │    │    └───────────────────► Pin 6 Arduino     │
│        │    └────────────────────────► Pin 7 Arduino     │
│        └─────────────────────────────► 5V                │
│                                                          │
│      ┌──────┐                                            │
│      │  ♪   │  BUZZER                                    │
│      └──────┘                                            │
│        │ (+) ──────────────────────────► Pin 9 Arduino   │
│        │ (-) ──────────────────────────► GND             │
└──────────────────────────────────────────────────────────┘
```

```
│                                                          │
│      🔴 LED INDICADOR                                    │
│      │                                                   │
│    [220Ω]──────────────────────────────► Pin 13 Arduino  │
│      │                                                   │
│      └──────────────────────────────────► GND            │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```
RESUMEN DE CONEXIONES:
═══════════════════════

SENSOR HC-SR04          ARDUINO
──────────────────      ───────
VCC   ──────────────►   5V
TRIG  ──────────────►   Pin 7
ECHO  ──────────────►   Pin 6
GND   ──────────────►   GND

BUZZER                  ARDUINO
──────────────────      ───────
(+) Positivo ───────►   Pin 9 (PWM ~)
(-) Negativo ───────►   GND

LED ROJO                ARDUINO
──────────────────      ───────
Ánodo (+) ─[220Ω]──►    Pin 13
Cátodo (-) ─────────►   GND
```

📋 PASO 4: Conexiones Paso a Paso Detalladas

```
text

PASO 4.1: Colocar el Arduino y Protoboard
──────────────────────────────────────────

1. Arrastra el Arduino UNO al área de trabajo
2. Arrastra el Breadboard y colócalo al lado del Arduino
3. Conecta la línea + del protoboard al pin 5V del Arduino (cable rojo)
4. Conecta la línea - del protoboard al pin GND del Arduino (cable negro)


PASO 4.2: Conectar el Sensor HC-SR04
──────────────────────────────────────

1. Arrastra el sensor ultrasónico al protoboard
2. Identifica los pines (de izquierda a derecha): VCC, TRIG, ECHO, GND

   Conexiones:
   • VCC → Línea + del protoboard (5V)
   • TRIG → Cable al Pin 7 del Arduino (cable amarillo)
   • ECHO → Cable al Pin 6 del Arduino (cable verde)
   • GND → Línea - del protoboard (GND)


PASO 4.3: Conectar el Buzzer
──────────────────────────────────────

1. Arrastra el Piezo buzzer al protoboard
2. El buzzer tiene dos patas:
   • Pata más larga o marcada (+) → Pin 9 del Arduino
   • Pata más corta (-) → Línea GND del protoboard
```

```
PASO 4.4: Conectar el LED Indicador
──────────────────────────────────────────

1. Coloca el LED rojo en el protoboard
2. Conecta una resistencia de 220Ω desde el ánodo (+)
3. Conecta la resistencia al Pin 13 del Arduino
4. Conecta el cátodo (-) al GND
```

📋 PASO 5: Programación con Bloques en Tinkercad

```
text

Para este proyecto, Tinkercad tiene limitaciones con bloques para
la melodía de Star Wars, así que usaremos el modo TEXTO.

1. Haz clic en "Code"
2. Selecciona "Text" en el menú desplegable
3. Copia el código que se muestra más adelante
```

Código en modo TEXTO para Tinkercad:

```
C++

// ═══════════════════════════════════════════════════════════════════
//        SENSOR ULTRASÓNICO + MELODÍA STAR WARS - TINKERCAD
// ═══════════════════════════════════════════════════════════════════

// Definición de pines
#define PIN_TRIG    7
#define PIN_ECHO    6
#define PIN_BUZZER  9
#define PIN_LED     13

// Distancia de activación en centímetros
#define DISTANCIA_ACTIVACION 4

// Definición de notas musicales (frecuencias en Hz)
#define NOTE_A4  440
#define NOTE_A5  880
#define NOTE_B4  494
#define NOTE_C5  523
#define NOTE_D5  587
#define NOTE_E5  659
#define NOTE_F5  698
#define NOTE_G5  784

// Variables globales
long duracion;
int distancia;

void setup() {
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_LED, OUTPUT);
  Serial.begin(9600);
}

void loop() {
```

```
  // Medir distancia
  distancia = medirDistancia();

  // Mostrar en monitor serial
  Serial.print("Distancia: ");
  Serial.print(distancia);
  Serial.println(" cm");

  // Si la distancia es menor a 4 cm
  if (distancia > 0 && distancia <= DISTANCIA_ACTIVACION) {
    digitalWrite(PIN_LED, HIGH);
    tocarStarWars();
  } else {
    digitalWrite(PIN_LED, LOW);
  }

  delay(100);
}

int medirDistancia() {
  // Limpiar el pin TRIG
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);

  // Enviar pulso de 10 microsegundos
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);

  // Leer el pulso de respuesta
  duracion = pulseIn(PIN_ECHO, HIGH);

  // Calcular distancia
  int dist = duracion * 0.034 / 2;

  return dist;
}

void tocarStarWars() {
  // Tema principal simplificado de Star Wars
  tone(PIN_BUZZER, NOTE_A4, 500);
  delay(500);
  tone(PIN_BUZZER, NOTE_A4, 500);
  delay(500);
  tone(PIN_BUZZER, NOTE_A4, 500);
  delay(500);
  tone(PIN_BUZZER, NOTE_F5, 350);
  delay(350);
  tone(PIN_BUZZER, NOTE_C5, 150);
  delay(150);
  tone(PIN_BUZZER, NOTE_A4, 500);
  delay(500);
  tone(PIN_BUZZER, NOTE_F5, 350);
  delay(350);
  tone(PIN_BUZZER, NOTE_C5, 150);
  delay(150);
  tone(PIN_BUZZER, NOTE_A4, 1000);
  delay(1000);
}
```

📋 PASO 6: Ejecutar Simulación en Tinkercad

text

```
1. Haz clic en "Start Simulation"
2. Acerca un objeto al sensor (haz clic y arrastra algo cerca)
3. En Tinkercad, puedes hacer clic en el sensor y ajustar la
   distancia manualmente
4. Cuando la distancia sea ≤ 4 cm:
   • El LED se encenderá
   • El buzzer tocará la melodía de Star Wars
```

🔧 PARTE 3: PROYECTO CON ARDUINO MEGA REAL

🛠️ Lista de Materiales

Componente	Cantidad	Descripción
Arduino MEGA 2560	1	Placa controladora
Sensor HC-SR04	1	Sensor ultrasónico
Buzzer activo/pasivo	1	Preferible pasivo para tonos
LED Rojo 5mm	1	Indicador visual
Resistencia 220Ω	1	Para el LED
Protoboard	1	Para conexiones
Cables Dupont	10+	Macho-Macho
Cable USB	1	Para programar

📋 PASO 1: Diagrama de Conexiones para Arduino MEGA

text

```
                    ARDUINO MEGA 2560
┌──────────────────────────────────────────────────────────┐
│                                                          │
│   ┌──────────────────────────────────────────────┐       │
│   │ DIGITAL PINS                                 │       │
│   │ 0  1  2  3  4  5  6  7  8  9  10 11 12 13    │       │
│   │                                              │       │
│   │                                              │       │
│   │                                              
```

VISTA DEL PROTOBOARD CON CONEXIONES:

```
┌──────────────────────────────────────────────────────────────────────────┐
│   -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  (GND)         │
│   +  +  +  +  +  +  +  +  +  +  +  +  +  +  +  +  +  +  +  (5V)          │
│                                                                          │
│   Fila 1:                                                                │
│            ┌──────────────────────────┐                                  │
│            │  ○     ○     ○     ○     │   ← Sensor HC-SR04               │
│            │ VCC  TRIG  ECHO  GND     │                                  │
│            └──────────────────────────┘                                  │
│              │      │      │      │                                      │
│              ▼      │      │      ▼                                      │
│             5V      │      │     GND                                     │
│                     │      │                                             │
│   Fila 10:          ▼      ▼                                             │
│            Cable    Cable                                                │
│            amarillo verde                                                │
│            (Pin 7)  (Pin 6)                                              │
│                                                                          │
│   Fila 15:                                                               │
│            ┌──────┐                                                      │
│            │  ♪   │   ← Buzzer                                           │
│            └──────┘                                                      │
│                │ (+) ──────────▶ Pin 9                                   │
│                │ (-) ──────────▶ GND                                     │
│                                                                          │
│   Fila 20:     🔴  ← LED                                                 │
│                │                                                         │
│            [220Ω] ──────────▶ Pin 13                                     │
│                │                                                         │
│                └──────────────▶ GND                                      │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

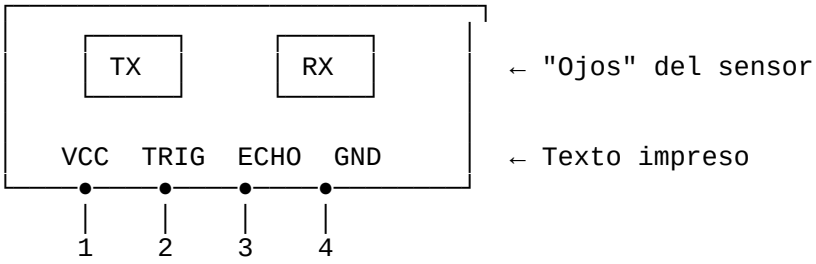
📋 PASO 2: Montaje Físico Detallado

text

```
═══════════════════════════════════════════════════════════════════════════
PASO 2.1: PREPARAR LA ALIMENTACIÓN DEL PROTOBOARD
═══════════════════════════════════════════════════════════════════════════

    1. Conecta un cable ROJO desde el pin 5V del Arduino MEGA
       a la línea (+) del protoboard

    2. Conecta un cable NEGRO desde el pin GND del Arduino MEGA
       a la línea (-) del protoboard

    Resultado:

    Arduino MEGA                Protoboard
    ┌──────────┐
    │    5V  ●─┼── Rojo ──▶ │ + + + + + + + + │
    │   GND  ●─┼── Negro ─▶ │ - - - - - - - - │
    └──────────┘

═══════════════════════════════════════════════════════════════════════════
PASO 2.2: CONECTAR EL SENSOR HC-SR04
═══════════════════════════════════════════════════════════════════════════
```

El sensor tiene 4 pines. Insértalo en el protoboard:

Vista frontal del sensor:

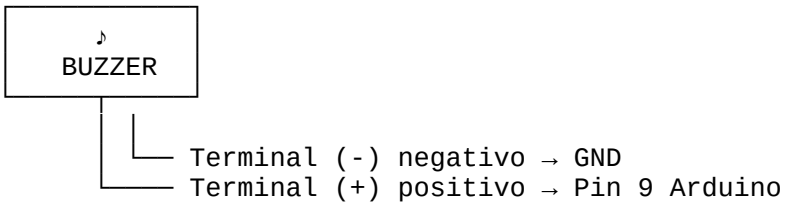


Conexiones:

Pin 1 (VCC) → Cable a línea (+) del protoboard
Pin 2 (TRIG) → Cable AMARILLO al Pin 7 del Arduino
Pin 3 (ECHO) → Cable VERDE al Pin 6 del Arduino
Pin 4 (GND) → Cable a línea (-) del protoboard

PASO 2.3: CONECTAR EL BUZZER

El buzzer tiene 2 terminales (algunos tienen marca + en el positivo):

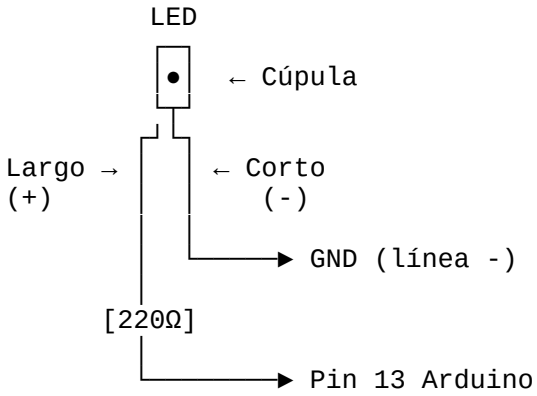


⚠️ IMPORTANTE: Usa un buzzer PASIVO para poder reproducir diferentes tonos. Un buzzer ACTIVO solo emite un tono fijo.

Cómo identificarlos:
- PASIVO: No suena al conectar 5V directamente
- ACTIVO: Suena inmediatamente al conectar 5V

PASO 2.4: CONECTAR EL LED INDICADOR

El LED indica visualmente cuando se detecta un objeto cercano:



📋 PASO 3: Código Completo para Arduino IDE

C++

```
/*
 * ═══════════════════════════════════════════════════════════════════════
 *
 *    ▄▄▄
```

```
#define DISTANCIA_MAXIMA      400  // Distancia máxima del sensor (400 cm)


// ═══════════════════════════════════════════════════════════════════════════
// SECCIÓN 3: DEFINICIÓN DE NOTAS MUSICALES
// ═══════════════════════════════════════════════════════════════════════════
// Cada nota musical tiene una frecuencia específica en Hertz (Hz)
// Estas frecuencias producen las notas que escuchamos

#define NOTE_B0  31
#define NOTE_C1  33
#define NOTE_CS1 35
#define NOTE_D1  37
#define NOTE_DS1 39
#define NOTE_E1  41
#define NOTE_F1  44
#define NOTE_FS1 46
#define NOTE_G1  49
#define NOTE_GS1 52
#define NOTE_A1  55
#define NOTE_AS1 58
#define NOTE_B1  62
#define NOTE_C2  65
#define NOTE_CS2 69
#define NOTE_D2  73
#define NOTE_DS2 78
#define NOTE_E2  82
#define NOTE_F2  87
#define NOTE_FS2 93
#define NOTE_G2  98
#define NOTE_GS2 104
#define NOTE_A2  110
#define NOTE_AS2 117
#define NOTE_B2  123
#define NOTE_C3  131
#define NOTE_CS3 139
#define NOTE_D3  147
#define NOTE_DS3 156
#define NOTE_E3  165
#define NOTE_F3  175
#define NOTE_FS3 185
#define NOTE_G3  196
#define NOTE_GS3 208
#define NOTE_A3  220
#define NOTE_AS3 233
#define NOTE_B3  247
#define NOTE_C4  262    // DO central
#define NOTE_CS4 277
#define NOTE_D4  294    // RE
#define NOTE_DS4 311
#define NOTE_E4  330    // MI
#define NOTE_F4  349    // FA
#define NOTE_FS4 370
#define NOTE_G4  392    // SOL
#define NOTE_GS4 415
#define NOTE_A4  440    // LA (nota de referencia estándar)
#define NOTE_AS4 466
#define NOTE_B4  494    // SI
#define NOTE_C5  523    // DO
#define NOTE_CS5 554
#define NOTE_D5  587    // RE
#define NOTE_DS5 622
#define NOTE_E5  659    // MI
#define NOTE_F5  698    // FA
```

```
#define NOTE_FS5 740
#define NOTE_G5  784    // SOL
#define NOTE_GS5 831
#define NOTE_A5  880    // LA
#define NOTE_AS5 932
#define NOTE_B5  988    // SI
#define NOTE_C6  1047
#define NOTE_CS6 1109
#define NOTE_D6  1175
#define NOTE_DS6 1245
#define NOTE_E6  1319
#define NOTE_F6  1397
#define NOTE_FS6 1480
#define NOTE_G6  1568
#define NOTE_GS6 1661
#define NOTE_A6  1760
#define NOTE_AS6 1865
#define NOTE_B6  1976
#define NOTE_C7  2093
#define NOTE_CS7 2217
#define NOTE_D7  2349
#define NOTE_DS7 2489
#define NOTE_E7  2637
#define NOTE_F7  2794
#define NOTE_FS7 2960
#define NOTE_G7  3136
#define NOTE_GS7 3322
#define NOTE_A7  3520
#define NOTE_AS7 3729
#define NOTE_B7  3951
#define NOTE_C8  4186
#define NOTE_CS8 4435
#define NOTE_D8  4699
#define NOTE_DS8 4978
#define REST      0     // Silencio (sin sonido)


// ═══════════════════════════════════════════════════════════════════════
// SECCIÓN 4: MELODÍA DE STAR WARS (TEMA PRINCIPAL)
// ═══════════════════════════════════════════════════════════════════════
// La melodía se define como un array de pares: nota y duración
// Las duraciones: 1=redonda, 2=blanca, 4=negra, 8=corchea, 16=semicorchea

// Tempo de la canción (beats por minuto)
int tempo = 108;

// Array de la melodía: nota, duración, nota, duración...
int melody[] = {

  // Compás 1-4: Fanfarria de apertura
  NOTE_AS4, 8, NOTE_AS4, 8, NOTE_AS4, 8,
  NOTE_F5, 2, NOTE_C6, 2,

  NOTE_AS5, 8, NOTE_A5, 8, NOTE_G5, 8, NOTE_F6, 2, NOTE_C6, 4,
  NOTE_AS5, 8, NOTE_A5, 8, NOTE_G5, 8, NOTE_F6, 2, NOTE_C6, 4,

  NOTE_AS5, 8, NOTE_A5, 8, NOTE_AS5, 8, NOTE_G5, 2, NOTE_C5, 8, NOTE_C5, 8,
NOTE_C5, 8,
  NOTE_F5, 2, NOTE_C6, 2,

  NOTE_AS5, 8, NOTE_A5, 8, NOTE_G5, 8, NOTE_F6, 2, NOTE_C6, 4,

  NOTE_AS5, 8, NOTE_A5, 8, NOTE_G5, 8, NOTE_F6, 2, NOTE_C6, 4,
  NOTE_AS5, 8, NOTE_A5, 8, NOTE_AS5, 8, NOTE_G5, 2, NOTE_C5, -8, NOTE_C5, 16,
```

```
  // Parte más tranquila
  NOTE_D5, -4, NOTE_D5, 8, NOTE_AS5, 8, NOTE_A5, 8, NOTE_G5, 8, NOTE_F5, 8,
  NOTE_F5, 8, NOTE_G5, 8, NOTE_A5, 8, NOTE_G5, 4, NOTE_D5, 8, NOTE_E5, 4,
NOTE_C5, -8, NOTE_C5, 16,

  NOTE_D5, -4, NOTE_D5, 8, NOTE_AS5, 8, NOTE_A5, 8, NOTE_G5, 8, NOTE_F5, 8,

  NOTE_C6, -8, NOTE_G5, 16, NOTE_G5, 2, REST, 8, NOTE_C5, 8,

  // Repetición del tema
  NOTE_D5, -4, NOTE_D5, 8, NOTE_AS5, 8, NOTE_A5, 8, NOTE_G5, 8, NOTE_F5, 8,
  NOTE_F5, 8, NOTE_G5, 8, NOTE_A5, 8, NOTE_G5, 4, NOTE_D5, 8, NOTE_E5, 4,
NOTE_C5, -8, NOTE_C5, 16,

  NOTE_D5, -4, NOTE_D5, 8, NOTE_AS5, 8, NOTE_A5, 8, NOTE_G5, 8, NOTE_F5, 8,

  NOTE_C6, -8, NOTE_G5, 16, NOTE_G5, 2, REST, 8, NOTE_C5, 8,

  // Final
  NOTE_F5, 4, NOTE_AS5, 8, NOTE_AS5, 8, NOTE_AS5, 8, NOTE_AS5, 8, NOTE_C6, 8,
  NOTE_AS5, 4, NOTE_A5, 4, NOTE_A5, 8, NOTE_A5, 8, NOTE_C6, 8, NOTE_AS5, 8,
  NOTE_G5, 4, NOTE_F5, 8, NOTE_F5, 8, NOTE_AS5, 8, NOTE_G5, 8, NOTE_C6, 2

};

// Calcular el número de notas
int notes = sizeof(melody) / sizeof(melody[0]) / 2;

// Calcular la duración de una nota completa (redonda) en ms
int wholenote = (60000 * 4) / tempo;


// ═══════════════════════════════════════════════════════════════════════════
// SECCIÓN 5: VARIABLES GLOBALES
// ═══════════════════════════════════════════════════════════════════════════

long duracion;           // Duración del pulso del sensor
int distancia;           // Distancia calculada en cm
bool melodiaEnCurso = false;  // Flag para saber si la melodía está sonando


// ═══════════════════════════════════════════════════════════════════════════
// SECCIÓN 6: FUNCIÓN SETUP - CONFIGURACIÓN INICIAL
// ═══════════════════════════════════════════════════════════════════════════

void setup() {

  // Configurar pines del sensor ultrasónico
  pinMode(PIN_TRIG, OUTPUT);   // TRIG es salida (envía el pulso)
  pinMode(PIN_ECHO, INPUT);    // ECHO es entrada (recibe el eco)

  // Configurar pin del buzzer
  pinMode(PIN_BUZZER, OUTPUT);

  // Configurar pin del LED
  pinMode(PIN_LED, OUTPUT);

  // Iniciar comunicación serial para debug
  Serial.begin(9600);

  // Mensaje de bienvenida
  Serial.println("╔═══════════════════════════════════════════════╗");
  Serial.println("║      SENSOR ULTRASÓNICO + STAR WARS           ║");
```

```
  Serial.println("│     Acerca tu mano a menos de 4 cm            │");
  Serial.println("└──────────────────────────────────────────────┘");
  Serial.println("");

  // LED de inicio (parpadeo)
  for (int i = 0; i < 3; i++) {
    digitalWrite(PIN_LED, HIGH);
    delay(200);
    digitalWrite(PIN_LED, LOW);
    delay(200);
  }
}


// ═══════════════════════════════════════════════════════════════════════
// SECCIÓN 7: FUNCIÓN LOOP - BUCLE PRINCIPAL
// ═══════════════════════════════════════════════════════════════════════

void loop() {

  // Medir la distancia usando el sensor ultrasónico
  distancia = medirDistancia();

  // Mostrar la distancia en el Monitor Serial
  Serial.print("📏 Distancia: ");
  Serial.print(distancia);
  Serial.println(" cm");

  // Verificar si hay un objeto dentro del rango de activación
  if (distancia > 0 && distancia <= DISTANCIA_ACTIVACION) {

    // Encender LED indicador
    digitalWrite(PIN_LED, HIGH);

    // Mostrar mensaje de activación
    Serial.println("🎵 ¡OBJETO DETECTADO! Reproduciendo Star Wars...");
    Serial.println("──────────────────────────────────────────────");

    // Reproducir la melodía de Star Wars
    tocarStarWars();

    // Esperar un momento antes de volver a detectar
    delay(500);

  } else {
    // No hay objeto cerca, apagar LED
    digitalWrite(PIN_LED, LOW);
  }

  // Pequeña pausa entre mediciones
  delay(100);
}


// ═══════════════════════════════════════════════════════════════════════
// SECCIÓN 8: FUNCIÓN PARA MEDIR DISTANCIA
// ═══════════════════════════════════════════════════════════════════════
/*
 * Esta función implementa el proceso de medición del sensor HC-SR04:
 *
 * 1. Envía un pulso ultrasónico
 * 2. Mide el tiempo que tarda en regresar el eco
 * 3. Calcula la distancia basándose en la velocidad del sonido
 *
```

```
 * Retorna: La distancia en centímetros (int)
 */

int medirDistancia() {

  // PASO 1: Asegurar que TRIG esté en LOW
  // Esto "limpia" el pin antes de enviar el pulso
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);  // Esperar 2 microsegundos

  // PASO 2: Enviar pulso de 10 microsegundos
  // Este pulso activa el emisor ultrasónico del sensor
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);  // Mantener HIGH por 10 µs
  digitalWrite(PIN_TRIG, LOW);

  // PASO 3: Leer la duración del pulso de respuesta en ECHO
  // pulseIn() mide cuánto tiempo está el pin en HIGH
  // El timeout de 30000 µs previene bloqueos si no hay eco
  duracion = pulseIn(PIN_ECHO, HIGH, 30000);

  // PASO 4: Calcular la distancia
  // Fórmula: distancia = (tiempo * velocidad del sonido) / 2
  // Velocidad del sonido = 343 m/s = 0.0343 cm/µs
  // Se divide entre 2 porque el sonido viaja ida y vuelta

  int dist = (duracion * 0.0343) / 2;

  // Validar el resultado
  if (dist <= 0 || dist > DISTANCIA_MAXIMA) {
    return 999;  // Valor que indica "fuera de rango"
  }

  return dist;
}


// ═══════════════════════════════════════════════════════════════════════
// SECCIÓN 9: FUNCIÓN PARA REPRODUCIR STAR WARS
// ═══════════════════════════════════════════════════════════════════════
/*
 * Esta función reproduce el tema principal de Star Wars
 * usando la función tone() de Arduino
 *
 * Recorre el array de melodía y toca cada nota con su duración
 */

void tocarStarWars() {

  int divider = 0;
  int noteDuration = 0;

  // Recorrer todas las notas de la melodía
  for (int thisNote = 0; thisNote < notes * 2; thisNote = thisNote + 2) {

    // Calcular la duración de cada nota
    divider = melody[thisNote + 1];

    if (divider > 0) {
      // Nota regular
      noteDuration = (wholenote) / divider;
    } else if (divider < 0) {
      // Nota con puntillo (duración aumentada 50%)
      noteDuration = (wholenote) / abs(divider);
```

```
      noteDuration *= 1.5;
    }

    // Reproducir la nota
    // Usamos 90% de la duración para dejar un pequeño silencio entre notas
    tone(PIN_BUZZER, melody[thisNote], noteDuration * 0.9);

    // Esperar la duración de la nota
    delay(noteDuration);

    // Detener el tono
    noTone(PIN_BUZZER);
  }
}


// ═══════════════════════════════════════════════════════════════════════
// FIN DEL PROGRAMA
// ═══════════════════════════════════════════════════════════════════════
```

📋 PASO 4: Instrucciones para Cargar el Código

```text
═══════════════════════════════════════════════════════════════════════
PROCEDIMIENTO DE CARGA EN ARDUINO MEGA
═══════════════════════════════════════════════════════════════════════

1. CONECTAR EL ARDUINO
   • Conecta el Arduino MEGA al PC con el cable USB
   • El LED de encendido debe encenderse

2. ABRIR ARDUINO IDE
   • Abre el programa Arduino IDE
   • Crea un nuevo sketch (Archivo → Nuevo)

3. COPIAR EL CÓDIGO
   • Selecciona todo el código de arriba
   • Pégalo en el Arduino IDE

4. CONFIGURAR LA PLACA

   Ve a: Herramientas → Placa → Arduino AVR Boards → Arduino Mega or Mega 2560

    ┌──────────────────────────────────────────┐
    │  Herramientas                            │
    │  ├── Placa: "Arduino Mega or Mega 2560" ◄──┤ Seleccionar
    │  ├── Procesador: "ATmega2560 (Mega 2560)"  │
    │  └── Puerto: "COM3" (o el que corresponda) │
    └──────────────────────────────────────────┘

5. SELECCIONAR EL PUERTO
   • Ve a: Herramientas → Puerto
   • Selecciona el puerto COM donde está conectado el Arduino
   • En Windows: COM3, COM4, etc.
   • En Mac: /dev/cu.usbmodem...
   • En Linux: /dev/ttyACM0 o /dev/ttyUSB0

6. VERIFICAR EL CÓDIGO
   • Haz clic en el botón ✓ (Verificar)
   • Espera a que aparezca "Compilación completada"
```

```
   • Si hay errores, revisa que copiaste todo el código

7. SUBIR EL CÓDIGO
   • Haz clic en el botón → (Subir)
   • Verás los LEDs TX/RX parpadear
   • Espera el mensaje "Subido" o "Done uploading"

8. PROBAR EL PROYECTO
   • Abre el Monitor Serial: Herramientas → Monitor Serial
   • Configúralo a 9600 baudios
   • Acerca tu mano al sensor
   • ¡Cuando esté a menos de 4 cm, sonará Star Wars!
```

📋 PASO 5: Uso del Monitor Serial

text

```
╔═══════════════════════════════════════════════════════════════╗
║                         MONITOR SERIAL                        ╒
╠═══════════════════════════════════════════════════════════════╣
│                                                               │
│   ╔═══════════════════════════════════════╗                   │
│   ║     SENSOR ULTRASÓNICO + STAR WARS    ╒                   │
│   ║     Acerca tu mano a menos de 4 cm    ╒                   │
│   ╚═══════════════════════════════════════╝                   │
│                                                               │
│   📏 Distancia: 45 cm                                         │
│   📏 Distancia: 32 cm                                         │
│   📏 Distancia: 18 cm                                         │
│   📏 Distancia: 8 cm                                          │
│   📏 Distancia: 5 cm                                          │
│   📏 Distancia: 3 cm                                          │
│   🎵 ¡OBJETO DETECTADO! Reproduciendo Star Wars...            │
│   ──────────────────────────────────────────                  │
│   📏 Distancia: 3 cm                                          │
│   📏 Distancia: 4 cm                                          │
│   📏 Distancia: 12 cm                                         │
│   📏 Distancia: 25 cm                                         │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

🔧 CÓDIGO ALTERNATIVO: Versión Simplificada

Si prefieres una versión más corta con solo el tema más reconocible:

C++

```
/*
 * VERSIÓN SIMPLIFICADA - Solo el inicio del tema de Star Wars
 */

#define PIN_TRIG    7
#define PIN_ECHO    6
#define PIN_BUZZER  9
#define PIN_LED     13
#define DISTANCIA_ACTIVACION 4

// Notas necesarias
```

```
#define NOTE_A4  440
#define NOTE_F5  698
#define NOTE_C5  523
#define NOTE_G5  784
#define NOTE_E5  659
#define NOTE_AS4 466
#define NOTE_A5  880

void setup() {
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_LED, OUTPUT);
  Serial.begin(9600);
  Serial.println("Star Wars Sensor - Listo!");
}

void loop() {
  int distancia = medirDistancia();

  Serial.print("Distancia: ");
  Serial.print(distancia);
  Serial.println(" cm");

  if (distancia > 0 && distancia <= DISTANCIA_ACTIVACION) {
    digitalWrite(PIN_LED, HIGH);
    Serial.println(">>> ACTIVADO! Reproduciendo melodía...");
    tocarMelodia();
    delay(1000);
  } else {
    digitalWrite(PIN_LED, LOW);
  }

  delay(100);
}

int medirDistancia() {
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);

  long duracion = pulseIn(PIN_ECHO, HIGH, 30000);
  int dist = (duracion * 0.0343) / 2;

  return (dist > 0 && dist < 400) ? dist : 999;
}

void tocarMelodia() {
  // Fanfarria inicial de Star Wars (muy reconocible)

  // "Bum bum bum..."
  nota(NOTE_A4, 500);
  nota(NOTE_A4, 500);
  nota(NOTE_A4, 500);

  // "Buuuum bum"
  nota(NOTE_F5, 350);
  nota(NOTE_C5, 150);

  // "Buuum"
  nota(NOTE_A4, 500);
```

```
  // "Buuum bum"
  nota(NOTE_F5, 350);
  nota(NOTE_C5, 150);

  // "Buuuuuuum"
  nota(NOTE_A4, 1000);

  delay(200);

  // Segunda parte
  nota(NOTE_E5, 500);
  nota(NOTE_E5, 500);
  nota(NOTE_E5, 500);

  nota(NOTE_F5, 350);
  nota(NOTE_C5, 150);

  nota(NOTE_G5, 500);

  nota(NOTE_F5, 350);
  nota(NOTE_C5, 150);

  nota(NOTE_A4, 1000);
}

void nota(int frecuencia, int duracion) {
  tone(PIN_BUZZER, frecuencia, duracion);
  delay(duracion + 50);  // Pequeña pausa entre notas
  noTone(PIN_BUZZER);
}
```

🔧 Solución de Problemas

text

```
┌──────────────────────────────────────────────────────────────────────┐
│                    SOLUCIÓN DE PROBLEMAS COMUNES                     │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  PROBLEMA: El sensor muestra distancias incorrectas o 0              │
│  ────────────────────────────────────────────────────────            │
│  ✓ Verificar conexiones de VCC y GND del sensor                      │
│  ✓ Confirmar que TRIG está en pin de salida                          │
│  ✓ Confirmar que ECHO está en pin de entrada                         │
│  ✓ Probar con otro sensor (puede estar defectuoso)                   │
│                                                                      │
│  PROBLEMA: El buzzer no suena                                        │
│  ────────────────────────────────────────────────────────            │
│  ✓ Verificar que es un buzzer PASIVO (no activo)                     │
│  ✓ Comprobar polaridad del buzzer                                    │
│  ✓ Probar con otro pin PWM (3, 5, 6, 9, 10, 11)                      │
│  ✓ Verificar que el buzzer funciona (conectar directo a 5V)          │
│                                                                      │
│  PROBLEMA: El buzzer suena muy bajo                                  │
│  ────────────────────────────────────────────────────────            │
│  ✓ Algunos buzzers pasivos son de bajo volumen                       │
│  ✓ Usar un amplificador o un buzzer más potente                      │
│  ✓ Verificar que el voltaje es de 5V                                 │
│                                                                      │
│  PROBLEMA: La melodía suena rara o desafinada                        │
│  ────────────────────────────────────────────────────────            │
```

```
✓ Ajustar el valor de 'tempo' en el código
✓ Verificar que las frecuencias de las notas son correctas
✓ El buzzer debe ser de tipo pasivo

PROBLEMA: Arduino no es detectado por el PC
──────────────────────────────────────────────────────────
✓ Instalar drivers CH340 o FTDI según el modelo
✓ Probar otro cable USB
✓ Probar otro puerto USB

PROBLEMA: La distancia mínima no funciona bien
──────────────────────────────────────────────────────────
✓ El HC-SR04 tiene distancia mínima de ~2 cm
✓ Ajustar DISTANCIA_ACTIVACION a 5 o 6 cm
✓ Superficie reflectante mejora la detección
```

📊 Resumen del Proyecto

text

```
╔═══════════════════════════════════════════════════════════════════╗
╔═══════════════════════════════════════════════════════════════════╗
                          RESUMEN DEL PROYECTO
╚═══════════════════════════════════════════════════════════════════╝
╚═══════════════════════════════════════════════════════════════════╝

🎯 OBJETIVO:
   Crear un sistema que detecte objetos cercanos y reproduzca
   la melodía de Star Wars cuando algo está a menos de 4 cm

📦 COMPONENTES USADOS:
   • Arduino MEGA 2560
   • Sensor ultrasónico HC-SR04
   • Buzzer pasivo
   • LED indicador

🔧 CONCEPTOS APRENDIDOS:
   • Funcionamiento del sensor ultrasónico (SONAR)
   • Cálculo de distancia con ultrasonido
   • Uso de la función tone() para generar sonidos
   • Reproducción de melodías con Arduino
   • Estructuras de control (if-else)
   • Funciones personalizadas

🔌 PINES UTILIZADOS:
   • Pin 7  → TRIG del sensor
   • Pin 6  → ECHO del sensor
   • Pin 9  → Buzzer (PWM)
   • Pin 13 → LED indicador
   • 5V     → Alimentación del sensor
   • GND    → Tierra común

⚡ FÓRMULA CLAVE:
   Distancia = (Tiempo × 0.0343) ÷ 2
```

🚀 Ideas para Expandir el Proyecto

```text
MEJORAS POSIBLES:

1. 🎵 MÚLTIPLES MELODÍAS
   Agregar más melodías (Imperial March, Cantina Band)
   y seleccionar aleatoriamente o según la distancia

2. 🌈 LEDS RGB
   Cambiar el color del LED según la distancia:
   • Verde: lejos
   • Amarillo: medio
   • Rojo: cerca

3. 📟 PANTALLA LCD
   Mostrar la distancia en una pantalla LCD I2C

4. 🔊 CONTROL DE VOLUMEN
   Agregar un potenciómetro para controlar el volumen

5. 📱 BLUETOOTH
   Enviar los datos de distancia a una app móvil
```